# Project Report Format

## 1. PROJECT TITLE

ResearchHub AI: Intelligent Research Paper Management and Analysis System using Agentic AI.

## 2. PROJECT DESCRIPTION

**System Overview**

ResearchHub AI is an intelligent web-based platform designed to help researchers efficiently discover, organize, and analyze academic research papers. The system integrates modern web technologies with agentic artificial intelligence to create a unified environment where users can search for papers, store them within project-specific workspaces, and interact with an AI assistant that provides contextual insights based on the content of the papers. By combining research data retrieval with AI-driven analysis, the system simplifies the literature review process and improves research productivity.

**Problem Domain**

The rapid growth of academic publications has created a major challenge for researchers who must review large volumes of literature to stay updated with developments in their fields. Traditional research workflows require manually searching multiple databases, downloading papers, organizing files, and reading through extensive documents. This process is time-consuming and often leads to missed insights or inefficient analysis. The proposed system addresses these challenges by providing an intelligent platform that automates many aspects of literature discovery and analysis.

**Core Technology Used**

The system is built using modern web and artificial intelligence technologies. The frontend is developed using React and TypeScript to provide an interactive user interface. The backend is implemented using FastAPI in Python to handle application logic and API interactions. The system integrates a Retrieval Augmented Generation architecture that uses vector embeddings and the Groq Llama 3.3 language model to analyze research papers and generate contextual responses. PostgreSQL with vector search capabilities is used for storing metadata and embeddings, while cloud storage is used to manage uploaded research papers.

**Key Capabilities**

The platform allows users to search research papers from external academic databases, import them into organized workspaces, upload their own documents, and interact with an AI assistant that can summarize and compare research findings. It also maintains contextual memory of user queries and research projects, enabling deeper analysis of multiple papers simultaneously. These capabilities significantly reduce the time required for literature review and research exploration.

**Target Users**

The system is designed for academic researchers, postgraduate students, university faculty members, and research professionals working in industrial research and development teams. It is also useful for students conducting literature surveys and for academic institutions seeking intelligent tools for research management.

**Real-world Relevance**

With the continuous expansion of scientific publications, researchers require advanced tools to efficiently analyze large volumes of knowledge. ResearchHub AI provides a practical solution that combines artificial intelligence and research management, enabling faster access to insights and improving the overall efficiency of research workflows.

## 3. APPLICATION SCENARIOS / USE CASES

**Academic Research Usage**

In academic environments, researchers and graduate students frequently conduct literature reviews to understand existing research and identify gaps in their field. Using ResearchHub AI, a researcher can search relevant papers, store them in project-specific workspaces, and ask the AI assistant to summarize findings or compare methodologies across multiple papers. This reduces manual effort and allows researchers to focus more on innovation and experimentation.

**Educational Usage**
In educational settings, students often struggle to interpret complex research papers when preparing assignments, thesis reports, or seminar presentations. The platform can assist students by providing simplified summaries of academic papers and highlighting key findings. Educators can also use the system to recommend important literature and help students understand relationships between different research works.

**Industrial Research Usage**
Research and development teams in industry must continuously monitor academic and technical publications to stay competitive. ResearchHub AI enables industry researchers to track emerging technologies, analyze multiple research papers quickly, and extract insights relevant to product development or innovation strategies.

**Independent Researcher Usage**
Independent researchers and professionals who are not affiliated with large institutions can use the platform to access organized research insights without needing multiple specialized tools. The system allows them to manage research papers efficiently and obtain AI-generated explanations that support their independent studies.

## 4. TECHNICAL ARCHITECTURE OVERVIEW

The system follows a multi-layer architecture designed to support scalable research analysis and efficient data processing. The architecture includes a frontend layer, backend layer, AI processing layer, and data storage layer that interact through structured APIs.

**Frontend Layer**
The frontend layer provides the graphical user interface through which users interact with the system. It is responsible for presenting the research dashboard, search interfaces, workspace management tools, and the AI chatbot interface. This layer is implemented using React and TypeScript to provide responsive and interactive web functionality.

**Backend Layer**
The backend layer handles the core application logic of the system. It manages user authentication, workspace operations, paper retrieval, document processing, and communication with AI services. The backend is implemented using FastAPI, which allows efficient API handling and asynchronous processing of user requests.

**Database Layer**
The database layer stores structured data required by the system, including user information, research workspaces, metadata of imported research papers, and conversation history. PostgreSQL is used as the primary relational database, and vector indexing is used to store embeddings that enable efficient semantic search across research documents.

**External APIs and Services**
The system integrates external research databases and AI services to obtain research information and generate insights. APIs such as ArXiv and Semantic Scholar are used to retrieve research paper metadata, while a large language model service is used to analyze document content and generate contextual responses.

**Deployment Environment**
The application is designed for deployment in a cloud-based environment. The frontend application can be hosted on modern web hosting platforms, while the backend services run on cloud servers or containerized environments. Cloud storage is used for managing uploaded research papers and system logs.

## 5. PREREQUISITES

### 5.1 Software Requirements
The development of the system requires standard software tools used in modern web application development. Developers should install an integrated development environment such as Visual Studio Code, along with Node.js for frontend development and Python for backend development. Package managers such as npm or pip are required to manage dependencies. Version control tools such as Git are also necessary for collaborative development. If cloud deployment is planned, access to cloud platforms such as AWS or similar hosting environments is required.

### 5.2 Libraries / Frameworks / Dependencies

The system uses several frameworks and libraries to implement its functionality. The frontend relies on React, TypeScript, and modern styling frameworks. The backend uses FastAPI and Python libraries for API development and data processing. Libraries for document parsing and vector embedding generation are used to process research papers. Database connectivity is handled through PostgreSQL drivers and vector database extensions. Additional libraries may be used for authentication, environment configuration, and asynchronous task processing.

### 5.3 Hardware Requirements

The system does not require specialized hardware for normal operation. A standard development computer with a modern processor, at least 8 to 16 gigabytes of memory, and sufficient storage capacity is adequate for development and testing. For production environments, cloud servers with sufficient computing resources are recommended to handle concurrent user requests and AI processing tasks.

## 6. PRIOR KNOWLEDGE REQUIRED

Before implementing the system, developers should have a basic understanding of modern software development concepts. Knowledge of programming languages such as Python and JavaScript is required for implementing backend and frontend components. Familiarity with web frameworks and RESTful API development is also necessary. Understanding database design and SQL fundamentals is important for managing application data. Basic knowledge of networking concepts and cloud deployment environments will help in deploying the system. Since the platform incorporates artificial intelligence, a conceptual understanding of machine learning, natural language processing, and retrieval augmented generation techniques will be beneficial.

## 7. PROJECT OBJECTIVES

### Technical Objectives

The primary technical objective of the project is to design and implement an intelligent platform that integrates research paper retrieval, document management, and AI-driven analysis within a unified system. The project aims to demonstrate how large language models and vector databases can be combined to enable contextual understanding of research documents.

### Performance Objectives

Another objective is to ensure that the system provides efficient search and analysis capabilities. The platform should retrieve research papers quickly and generate AI responses within a reasonable time frame, enabling researchers to interact with the system without significant delays.

### Deployment Objectives

The system is intended to be deployable in a scalable cloud environment so that it can support multiple concurrent users and large research datasets. The architecture is designed to allow easy integration of new APIs, AI models, and additional features in future development phases.

### Learning Outcomes

The project also aims to provide practical learning outcomes for developers by exposing them to modern technologies such as web application frameworks, API integration, cloud-based deployment, vector databases, and artificial intelligence systems. Through this project, developers gain experience in designing scalable systems and integrating advanced AI capabilities into real-world applications.

## 8. SYSTEM WORKFLOW

### 1. User Interaction

The workflow begins when a user accesses the ResearchHub AI platform through the web interface. After logging into the system, the user can create or select a research workspace and initiate actions such as searching for research papers, importing selected papers into a workspace, uploading PDF documents, or asking questions through the AI chatbot. The frontend interface built with React and TypeScript provides an interactive environment where users can manage research content and communicate with the AI assistant.

**2. Input Handling**
Once a user submits an action, such as entering a research topic or asking a question, the input is captured by the frontend application and sent to the backend server through HTTP requests. The FastAPI backend receives the request and validates it using authentication tokens and request schemas. For example, a search query triggers the paper search API, while a chatbot question is routed to the AI query endpoint.

**3. Processing Logic**
After receiving the request, the backend processes the input according to the requested operation. For paper search queries, the system communicates with external research databases such as arXiv and PubMed to retrieve relevant research paper metadata. For AI queries, the backend retrieves relevant research documents from the database and prepares contextual information required for analysis. The processing stage also includes tasks such as document parsing, embedding generation, and workspace management.

**4. Core System Execution**
In this stage, the system executes the core functionality of the platform. Imported research papers are processed by extracting textual content and generating vector embeddings using a sentence transformer model. These embeddings are stored in the vector-enabled PostgreSQL database to support semantic similarity search. When a user asks a question, the system retrieves the most relevant document segments using vector similarity search and constructs a context-aware prompt. This prompt is then sent to the Groq Llama 3.3 language model to generate an intelligent response based on the retrieved research content.

**5. Output Generation**
After processing the contextual prompt, the language model produces a structured response that explains the research findings, summarizes information from multiple papers, or compares methodologies depending on the user's query. The generated output is formatted by the backend to include relevant information such as explanations, summarized insights, and references to the source research papers.

**6. Response Delivery**
Finally, the generated response is sent back to the frontend interface through the backend API. The frontend displays the AI-generated answer within the chatbot interface, allowing users to review insights and continue interacting with the system. The conversation history is also stored in the database so that future queries within the same workspace can maintain contextual continuity. This completes the workflow cycle, enabling users to efficiently explore and analyze research literature through the platform.

## 9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

Milestone 1 focuses on identifying the problem domain, defining system requirements, and establishing the foundational design decisions for the ResearchHub AI platform. This stage ensures that the system architecture, technologies, and development approach align with the overall objective of creating an intelligent research paper management and analysis platform. Proper requirement analysis allows developers to clearly understand user needs and translate them into functional and technical specifications.

### 9.1 Problem Definition

The exponential growth of academic research publications has created a significant challenge for researchers who need to stay updated with new developments in their fields. Millions of research papers are published every year across various disciplines, making it difficult for researchers to efficiently locate, organize, and analyze relevant literature.

Traditional research workflows require manually searching multiple academic databases, downloading papers, organizing files locally, and reading extensive documents to identify useful insights. This process is time-consuming and often results in missed information or inefficient research analysis. Furthermore, existing research management tools mainly focus on reference storage or citation management but do not provide intelligent analysis capabilities.

The problem addressed by the ResearchHub AI system is the lack of an integrated platform that combines research discovery, document organization, and AI-powered contextual analysis. The goal of the system is to simplify literature

review processes by providing a unified environment where researchers can discover papers, organize them within structured workspaces, and interact with an AI assistant capable of summarizing and synthesizing research insights.

### 9.2 Functional Requirements

Functional requirements define the core features and operations that the system must perform to meet user needs.

The system must allow users to register and log in securely in order to access personalized research workspaces. Users should be able to search research papers using keywords through integrated academic databases such as arXiv and PubMed. The system must retrieve and display research paper metadata including titles, authors, abstracts, and source links.

Users should be able to import selected research papers into project-specific workspaces where they can manage and organize research materials. The system must support uploading PDF documents and extracting textual content from those documents for further analysis.

The platform must generate vector embeddings for research papers to enable semantic search and contextual analysis. Users should be able to interact with an AI chatbot that answers questions based on the content of imported papers using a Retrieval-Augmented Generation pipeline.

Additionally, the system should maintain conversation history and workspace context so that the AI assistant can provide more relevant and continuous responses during research discussions.

### 9.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance expectations of the system.

The system should provide responsive performance with minimal delay when searching research papers or generating AI responses. The architecture must support scalability so that the platform can handle increasing numbers of users and research documents without significant performance degradation.

Security is another important requirement. User authentication must be implemented using secure password hashing and token-based authentication mechanisms to protect user accounts and research data. Communication between the frontend and backend should be secured through appropriate protocols.

Reliability is also essential for a research platform. The system should maintain stable operation and handle errors gracefully when external APIs or services are temporarily unavailable. Data consistency must be ensured when storing research papers, embeddings, and conversation history in the database.

Finally, the system should be maintainable and extensible so that new features, AI models, or research APIs can be integrated in future development stages.

### 9.4 System Design Decisions

Several important design decisions were made during the system design phase to ensure efficiency, modularity, and scalability.

A multi-layer architecture was adopted to separate the frontend user interface, backend application logic, database storage, and AI processing components. This modular architecture allows each component to be developed, tested, and maintained independently.

A Retrieval-Augmented Generation (RAG) architecture was selected to enable contextual AI responses grounded in actual research papers. Instead of relying solely on a language model, the system retrieves relevant document segments using vector similarity search before generating an answer.

Vector embeddings were incorporated into the system to support semantic document retrieval. Research paper content is converted into high-dimensional vectors and stored in a vector-enabled database, allowing efficient similarity searches based on user queries.

The backend was designed using RESTful APIs to facilitate communication between the frontend application and backend services. This design approach allows the frontend interface to interact with multiple backend components including authentication, paper search services, workspace management, and AI inference modules.

### 9.5 Technology Stack Selection Justification

The selection of technologies for the ResearchHub AI system was based on performance requirements, scalability considerations, and compatibility with AI-based research analysis.

React and TypeScript were chosen for frontend development because they provide a flexible framework for building responsive and maintainable web interfaces. React's component-based architecture allows efficient management of complex user interfaces such as dashboards, search pages, and chatbot panels.

FastAPI was selected for backend development due to its high performance and support for asynchronous programming. It enables efficient API development and integrates well with Python-based AI and machine learning libraries.

PostgreSQL was chosen as the primary database system because it provides reliable relational data storage and supports vector extensions such as pgvector for storing embeddings used in semantic search operations.

For AI processing, the Groq Llama 3.3 language model was selected due to its strong natural language understanding capabilities and efficient inference performance. The system also uses sentence transformer models to generate vector embeddings for research documents.

Additional technologies such as Docker and containerized deployment environments were considered to simplify deployment and enable scalability across cloud-based infrastructure.

Together, this technology stack provides a robust foundation for building a scalable and intelligent research management platform.

```
┌──────────────────────────────────────────────────────────────────┐
│  React + TypeScript Frontend  (port 3000)                        │
│   Login · Search Papers · Workspaces · AI Chatbot                │
└──────────────────────────┬──────────────────────────────────────┘
                           │ HTTP / REST
┌──────────────────────────▼──────────────────────────────────────┐
│  FastAPI Backend  (port 8000)                                    │
│   /auth  /papers  /workspaces  /chat                             │
└──────────────────────────────────────────────────────────────────┘
    │                    │                      │
    ▼                    ▼                      ▼
PostgreSQL          pgvector                Groq API
(ResearchHub DB)    similarity search       Llama-3.3-70b
port 5432           (768-dim embeddings)    temperature 0.3
```

## 10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

### 10.1 Development Environment Setup

The development environment was configured to support both frontend and backend components of the system.

1) **Programming Languages:** Python for backend development and JavaScript/TypeScript for frontend development.
2) **IDE:** Visual Studio Code for coding, debugging, and version control integration.
3) **Runtime Environment:**
    a. Python 3.10 or higher
    b. Node.js version 18 or higher
4) **Version Control:** Git for collaborative development and source code management.
5) **Database Setup:** PostgreSQL with pgvector extension for storing research metadata and vector embeddings.
6) **Containerization:** Docker and Docker Compose were used to manage database services and ensure a consistent development environment.

### 10.2 Dependency Installation

Dependencies required for the system were installed separately for backend and frontend components.

Backend Dependencies (Python):

- FastAPI – API development framework
- Uvicorn – ASGI server for running FastAPI applications
- SQLAlchemy – Database ORM for PostgreSQL interaction
- Groq SDK – Integration with Llama 3.3 language model
- Sentence Transformers – Generation of vector embeddings
- Passlib and python-jose – User authentication and JWT handling
- PyMuPDF – PDF parsing and text extraction

Installation commands:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Frontend Dependencies (Node.js):

- React – User interface development
- TypeScript – Type-safe JavaScript development
- Tailwind CSS – UI styling framework
- Axios – API communication with backend

Installation command :

```
npm install
```

### 10.3 Project Structure Creation

```
researchhub-ai/
```

```
│
├── backend/
│   ├── app/
│   │   ├── routers/
│   │   ├── services/
│   │   ├── database/
│   │   ├── agents/
│   │   └── utils/
│   └── requirements.txt
│
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   └── hooks/
│   └── package.json
│
└── docker-compose.yml
```

**10.4 Configuration Setup**

1. System configuration was completed by defining environment variables and connecting system components.
2. **Environment File (.env):**
    a. GROQ_API_KEY – Access key for Groq Llama 3.3 AI model
    b. SECRET_KEY – Used for JWT authentication
    c. DATABASE_URL – PostgreSQL database connection string
    d. ACCESS_TOKEN_EXPIRE_MINUTES – Token expiration configuration
3. Example configuration:
4. GROQ_API_KEY=your_api_key
   SECRET_KEY=your_secret_key
   DATABASE_URL=postgresql://user:password@localhost:5432/ResearchHub
   ACCESS_TOKEN_EXPIRE_MINUTES=60
5. **CORS Configuration:** Enabled communication between frontend (port 3000) and backend (port 8000).
6. **Database Initialization:** Tables and pgvector extension were created automatically during backend startup.
7. **Docker Configuration:** Used to run PostgreSQL and related services through Docker containers.
8. After completing these steps, the development environment was fully prepared and ready for the implementation of the core system features in the next milestone.

# 11 MILESTONE 3: CORE SYSTEM DEVELOPMENT

## 11.1 Feature 1 – Implementation

### 11.1.1 Description

The first core feature implemented in the system is the **research paper search and import functionality**. This feature enables users to search for academic research papers from external research databases and import relevant papers into their personal research workspaces.

When a user enters a research topic or keyword, the system sends the query to external academic APIs such as **arXiv and PubMed** to retrieve research paper metadata. The retrieved results contain important information including the paper title, authors, abstract, publication source, and publication date.

Users can browse the search results through the frontend interface and select relevant papers to import into a workspace. Once a paper is imported, its metadata is stored in the database and associated with the selected workspace. The stored paper data can then be used for additional processing such as **embedding generation, vector indexing, and AI-based research analysis**.

This feature serves as the foundation of the system's research workflow, allowing users to build organized collections of research papers that can later be analyzed using the AI-powered research assistant

### 11.1.2 Code Explanation

The research paper search and import functionality is implemented through the **backend API layer developed using FastAPI**, along with frontend components built using **React and TypeScript**. The system follows a modular architecture where separate services handle search operations, data processing, and database storage.

When a user enters a search query in the frontend interface, the request is sent to the backend through a **REST API endpoint**. The backend processes the request and communicates with external academic databases such as arXiv and PubMed to retrieve research paper information.

The implementation of this feature involves the following steps:

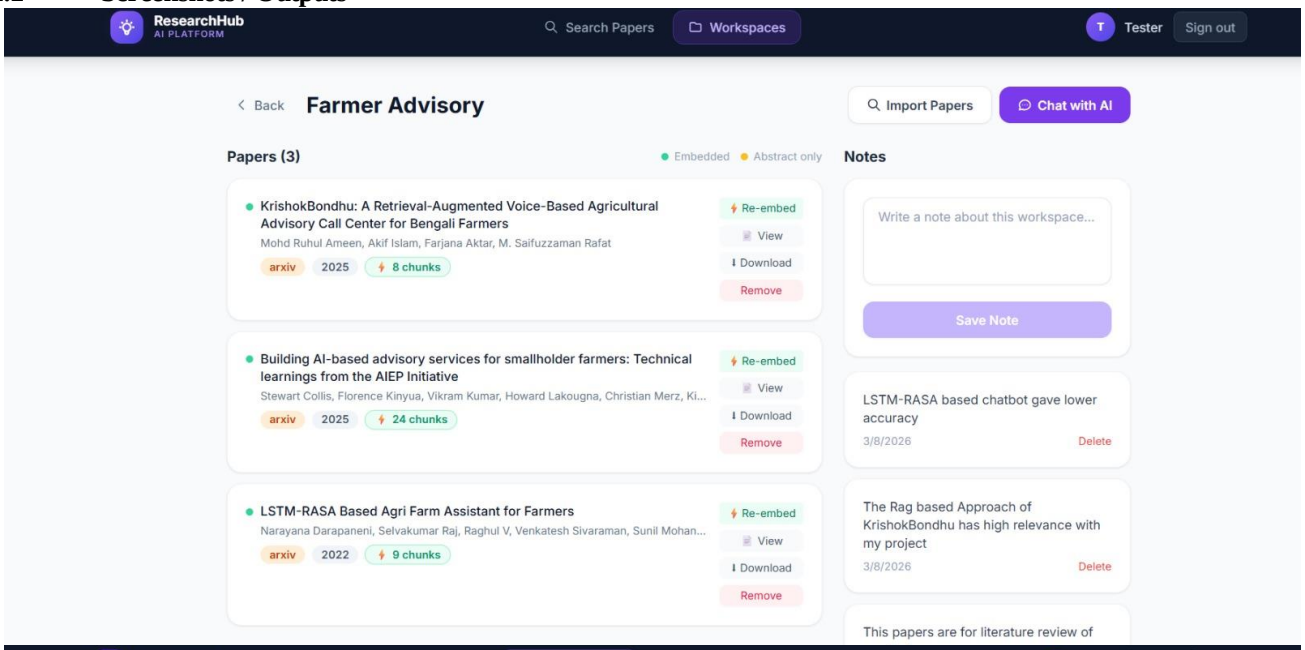
- **Search Query Processing**
  The frontend search component captures the user's keyword or research topic and sends it to the backend using an HTTP request.
- **API Endpoint Handling**
  The backend defines a search endpoint that receives the query and triggers the paper retrieval process.
- **External API Communication**
  The backend service sends requests to academic research APIs such as arXiv or PubMed. The responses from these APIs contain paper metadata including title, authors, abstract, and publication details.
- **Data Parsing and Formatting**
  The retrieved data is processed and converted into a standardized structure so that results from different research sources can be displayed consistently on the frontend interface.
- **Displaying Search Results**
  The formatted results are returned to the frontend, where they are presented as a list of research papers along with their metadata.
- **Paper Import Functionality**
  When a user selects a paper for import, another backend API endpoint is triggered. This endpoint stores the selected paper metadata in the PostgreSQL database and associates it with the selected workspace.

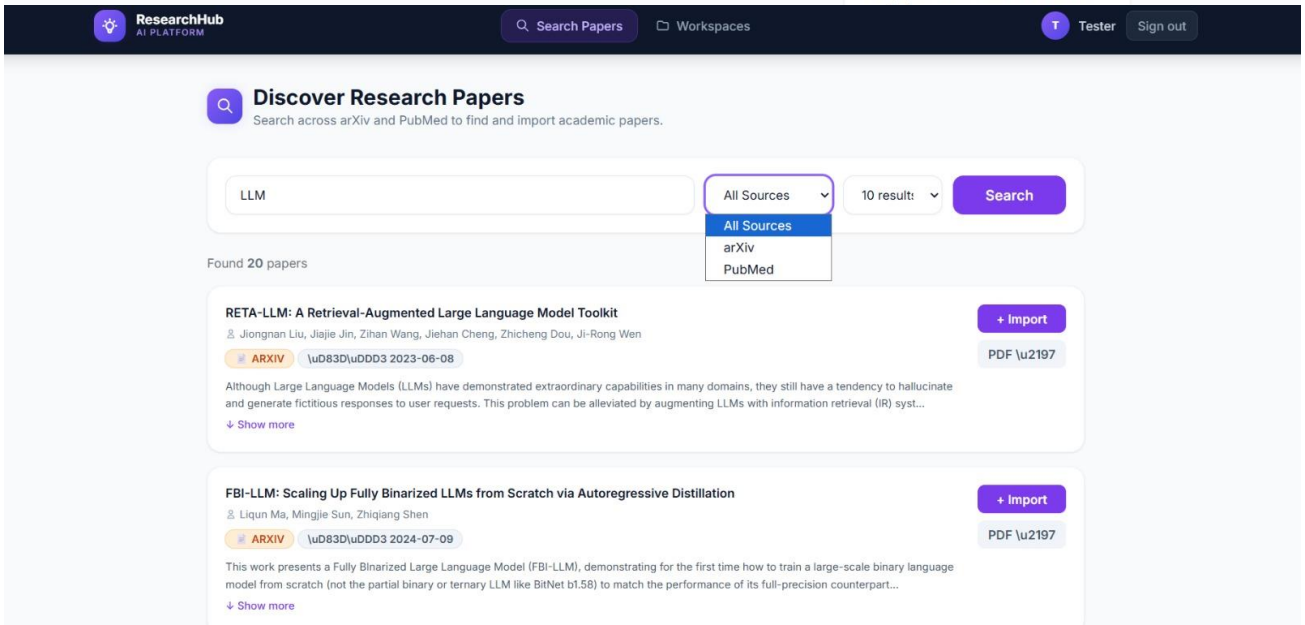
The implementation involves several key components:

- **FastAPI Router** – Handles incoming search and import API requests.
- **Search Service Module** – Communicates with external research APIs and processes their responses.
- **Database Models (SQLAlchemy)** – Stores paper metadata and maintains relationships between users, workspaces, and papers.
- **React Frontend Components** – Provide the search interface and display retrieved research papers.

This modular architecture ensures that the search functionality remains scalable and allows additional research databases to be integrated into the system in the future

### 11.1.2 Screenshots / Outputs




## 11.2 Feature 2 – Implementation

### 11.2.1 Description

The second core feature implemented in the system is the workspace management functionality. This feature allows users to organize and manage research papers within structured workspaces based on different research topics or projects.

A workspace acts as a dedicated environment where users can store imported research papers and maintain organized collections of research material. This helps researchers categorize papers related to specific domains, making it easier to perform literature reviews and analysis.

Users can create multiple workspaces depending on their research needs. After searching and selecting research papers, users can import them into a selected workspace. Once imported, the paper metadata is linked to that workspace in the database, allowing users to easily retrieve and manage the stored papers.

The workspace management system also maintains relationships between users, workspaces, and research papers, ensuring that each user can securely manage their own research collections.

This feature plays an important role in organizing research content and provides the necessary structure for the AI research assistant to analyze papers within a specific research context.

### 11.2.2 Code Explanation

The workspace management functionality is implemented using the FastAPI backend, PostgreSQL database, and React frontend components. The system handles operations such as workspace creation, retrieval, and paper association through dedicated API endpoints.

When a user performs an operation such as creating a workspace or viewing existing workspaces, the frontend sends a request to the backend server. The backend processes the request and performs the required database operations.

The implementation includes the following processes:

• Workspace Creation
When a user creates a new workspace, the frontend sends a request to the backend API. The backend validates the request and stores the workspace information in the database, linking it with the user account.

• Workspace Retrieval
The backend retrieves the list of workspaces associated with the logged-in user. This information is returned to the frontend and displayed in the workspace dashboard.

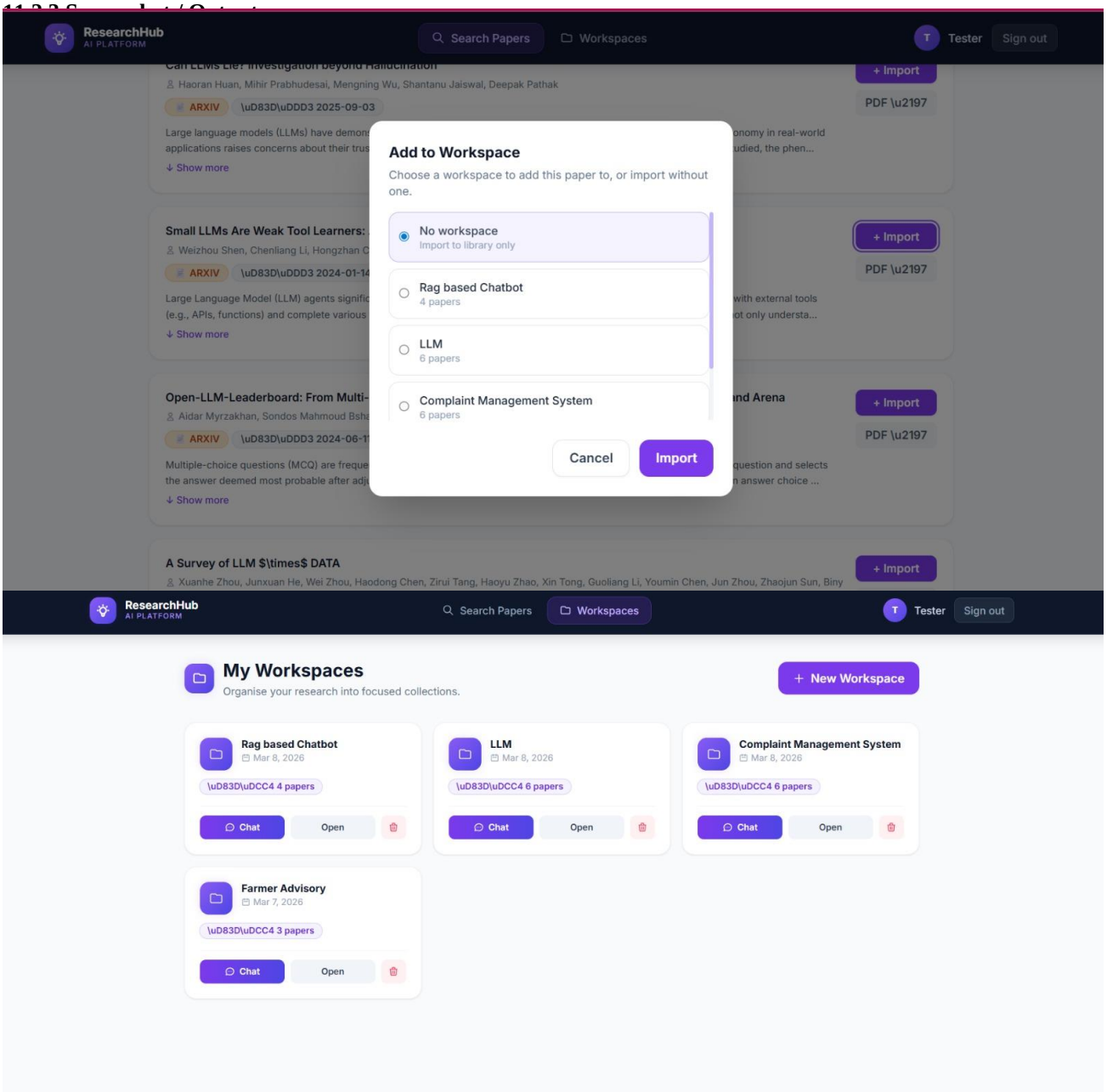
• Paper Association with Workspace
When a research paper is imported, the system associates the paper record with the selected workspace using relational database tables.

• Workspace Data Management
The system allows users to view papers stored within each workspace and manage their research collections efficiently.

The implementation involves several key components:

- Workspace Router (FastAPI) – Handles API endpoints related to workspace operations such as creation and retrieval.
- Database Models (SQLAlchemy) – Define relationships between users, workspaces, and research papers.
- Workspace Service Module – Processes business logic related to workspace management.
- React Frontend Components – Provide the workspace dashboard interface and display papers within each workspace.

This modular design ensures that workspace operations remain efficient and scalable while maintaining proper data organization within the system.




## 11.3 Feature 3 – Implementation

### 11.3.1 Description

The third core feature implemented in the system is the AI-powered research assistant, which enables users to analyze and interact with research papers through an intelligent chatbot interface. This feature uses a Retrieval Augmented Generation (RAG) approach to generate responses based on the content of research papers stored in the system.

Once research papers are imported into a workspace, their content becomes available for AI-based analysis. Users can ask research-related questions through the chatbot interface, such as summarizing a paper, explaining key concepts, or comparing ideas across multiple papers.

When a user submits a query, the system retrieves relevant information from stored research papers and provides this information as context to the AI model. The model then generates a response that is grounded in the retrieved research content. This approach improves the accuracy and reliability of responses compared to using a general language model alone.

This feature significantly reduces the time required for manual literature review and helps researchers quickly extract insights from multiple research papers.

### 11.3.2 Code Explanation

The AI research assistant is implemented through a multi-stage processing pipeline that combines document processing, vector embeddings, semantic search, and large language model reasoning.

When a user submits a query through the chatbot interface, the system performs several steps to generate a response.

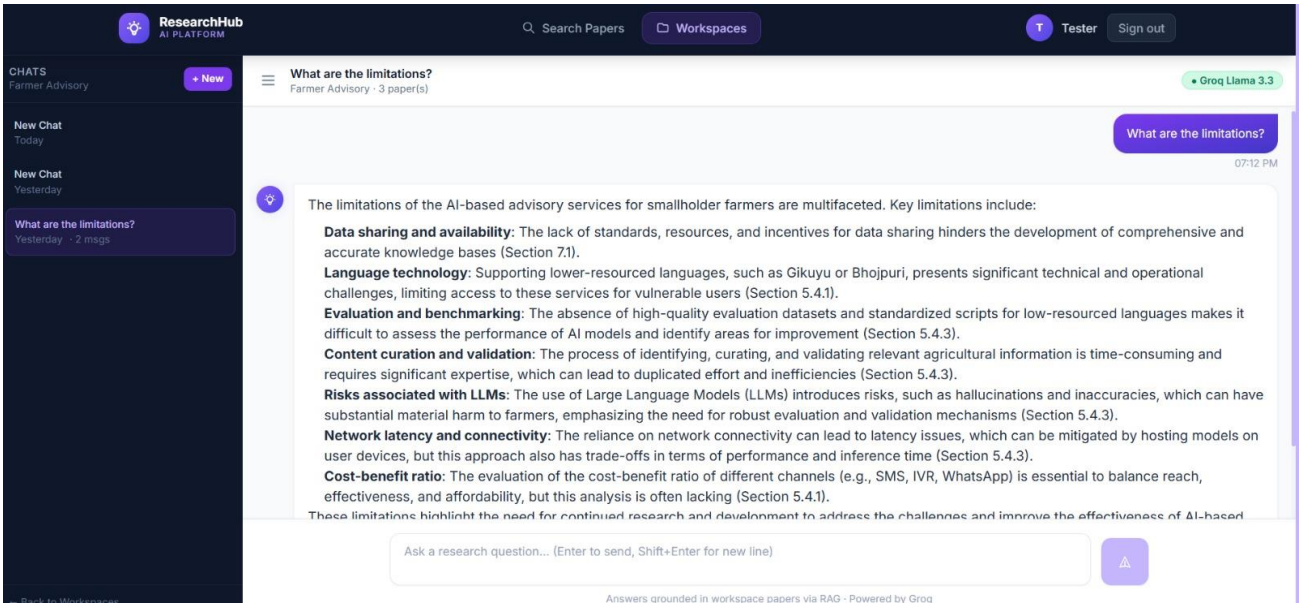
- Text Extraction and Processing
  When a research paper is imported, the system extracts the textual content and divides it into smaller segments known as text chunks. This allows the system to analyze large documents more efficiently.
- Embedding Generation
  Each text chunk is converted into a vector embedding using a sentence transformer model. These embeddings represent the semantic meaning of the text.
- Vector Storage
  The generated embeddings are stored in the PostgreSQL database using the pgvector extension, allowing efficient similarity search operations.
- Query Processing
  When a user submits a question, the system converts the query into a vector representation using the same embedding model.
- Similarity Search
  The system performs a vector similarity search to retrieve the most relevant text chunks from the stored research papers.
- AI Response Generation
  The retrieved text segments are provided as context to the Groq Llama 3.3 language model, which generates a response based on the research content.

The implementation involves the following key components:

- Chat Router (FastAPI) – Handles chatbot query requests from the frontend.
- Embedding Service – Generates vector embeddings for document text.
- Vector Search Module – Performs similarity search using pgvector.
- LLM Service – Communicates with the Groq API to generate responses.
- React Chat Interface – Allows users to interact with the AI assistant.

This pipeline ensures that AI responses are generated based on actual research paper content rather than general knowledge alone.

### 11.3.3 Screenshot/ Output



# 12 MILESTONE 4: INTEGRATION & OPTIMIZATION

## 12.1 Component Integration

Component integration ensures seamless communication between all layers of the ResearchHub AI system.

- **Frontend–Backend Integration:**
  The React frontend communicates with the FastAPI backend through RESTful APIs using HTTP requests.

- **Backend–Database Integration:**
  SQLAlchemy ORM is used to connect the backend with the PostgreSQL database for managing users, workspaces, papers, and chat history.

- **AI Integration:**
  The backend integrates the Groq Llama 3.3 model for generating contextual responses through the chatbot.

- **External API Integration:**
  Research paper metadata is retrieved from external sources such as **arXiv API** and **PubMed E-utilities API**.

- **Vector Search Integration:**
  The pgvector extension enables semantic similarity search by comparing vector embeddings generated from research papers.

These integrations allow the system to perform tasks such as paper retrieval, document analysis, and AI-assisted question answering within a unified workflow.

## 12.2 Performance Optimization

Several techniques were applied to improve system performance and response time.

- Asynchronous API Handling: FastAPI uses asynchronous request processing to handle multiple user requests efficiently.

- Vector Similarity Search: pgvector enables fast retrieval of relevant document embeddings.

- Efficient Embedding Storage: Research paper embeddings are stored once and reused for future queries.

- Background Processing: Embedding generation and document processing are handled as background tasks to reduce user waiting time.

- Optimized AI Parameters: The Groq Llama 3.3 model uses controlled parameters (e.g., temperature settings) to produce faster and consistent responses.

These optimizations help maintain low latency for search operations and AI responses

### 12.3 Security Enhancements

Security mechanisms were implemented to protect user data and system access.

- **Authentication:** Secure user login using **JWT (JSON Web Tokens)**.
- **Password Protection:** Passwords are hashed using **bcrypt** through the Passlib library.
- **Environment Variables:** Sensitive data such as API keys and database credentials are stored in .env configuration files.
- **CORS Protection:** Cross-Origin Resource Sharing policies restrict unauthorized access to backend APIs.
- **Secure API Communication:** Authentication tokens are required for accessing protected endpoints.

These security measures ensure that research data and user information remain protected.

### 12.4 Error Handling & Validation

Error handling and input validation mechanisms were implemented to improve system reliability.

- Input Validation: Request data is validated using Pydantic schemas in FastAPI.
- API Error Responses: Structured error messages are returned for invalid requests or failed operations.
- Exception Handling: Backend services include exception handling for database errors, API failures, and file processing issues.
- Fallback Mechanisms: When a research paper PDF is unavailable, the system falls back to using abstract metadata for analysis.
- Logging: System logs are generated to track errors and assist in debugging.

These mechanisms help maintain stable system behavior and provide clear feedback to users when errors occur.

## 13 MILESTONE 5: TESTING & VALIDATION

### 13.1 Test Cases

| Test Case ID | Input | Expected Output | Status |
|---|---|---|---|
| TC01 | User enters valid email and password | User successfully logs into the system | Passed |
| TC02 | User enters invalid login credentials | System displays authentication error message | Passed |
| TC03 | User searches for research papers using keyword | System returns relevant papers from academic APIs | Passed |
| TC04 | User imports a paper into a workspace | Paper metadata stored and displayed in workspace | Passed |
| TC05 | User asks a question in AI chatbot | System retrieves relevant paper content and generates response | Passed |

| Test Case ID | Input | Expected Output | Status |
|---|---|---|---|
| TC06 | User creates a new workspace | Workspace is successfully created and displayed | Passed |
| TC07 | System retrieves embeddings for query | Relevant document chunks returned for analysis | Passed |
| TC08 | User accesses database through Adminer | Database tables displayed successfully | Passed |

### 13.2 Unit Testing

Unit testing was conducted to verify the functionality of individual components within the system. Each module was tested independently to ensure that it performs its intended function correctly.

Key modules tested include:

- **Authentication Module**
  - Validation of user registration and login
  - Password hashing and JWT token generation
- **Search Engine Module**
  - API communication with arXiv and PubMed
  - Correct formatting of retrieved research paper data
- **Workspace Management Module**
  - Creation and retrieval of workspaces
  - Importing and storing research papers
- **AI Processing Module**
  - Text extraction and chunk generation
  - Embedding generation using Sentence Transformers
  - Vector similarity search using pgvector

Successful unit testing ensured that each module operates independently without errors.

### 13.3 Integration Testing

Integration testing was performed to verify that different system components communicate and function together correctly.

Major integrations tested include:

- Communication between **React frontend and FastAPI backend**
- Database connectivity between **FastAPI and PostgreSQL**
- API integration with **arXiv and PubM ed**
- AI processing pipeline involving **embedding generation, vector search, and LLM response generation**
- Communication between **Groq API and backend chatbot module**

The testing confirmed that data flows correctly across all layers of the system.

### 13.4 User Acceptance Testing

User acceptance testing (UAT) was conducted to evaluate the usability and functionality of the system from an end-user perspective.

Test users interacted with the platform and performed typical research tasks such as:

- Creating user accounts and logging into the system
- Searching for research papers
- Importing papers into workspaces
- Asking questions using the AI research assistant
- Navigating between workspaces and chat history

Feedback from users indicated that the system provided a **clear interface, accurate research responses, and efficient navigation** for managing research papers.

## 13.5 Performance Metrics

### 13.5.1 Accuracy

Accuracy measures how often the system retrieves relevant research information when responding to user queries. The RAG-based AI assistant demonstrated high accuracy by retrieving the most relevant document chunks through vector similarity search before generating responses using the LLM.

### 13.5.2 Precision

Precision measures the proportion of retrieved research content that is actually relevant to the user's query. The use of semantic embeddings significantly improves precision by ensuring that retrieved text segments are contextually related to the query.

### 13.5.3 Recall

Recall evaluates the system's ability to retrieve all relevant information related to a research query. The vector search mechanism ensures that multiple relevant document segments are retrieved, improving the overall recall of the AI assistant.

### 13.5.4 F1 Score

The F1 Score represents the harmonic mean of precision and recall, providing a balanced evaluation of the AI system's retrieval effectiveness.
By combining semantic embeddings with LLM reasoning, the ResearchHub AI system achieves balanced performance in retrieving and synthesizing research information.

### 13.5.5 Response Time

Response time measures how quickly the system processes user requests and generates results.
Key observations include:

- Paper search results are retrieved within a few seconds.
- AI chatbot responses are generated within a short processing time depending on query complexity.
- Backend API endpoints respond efficiently due to asynchronous FastAPI architecture.

### 13.5.6 Load Handling

Load handling tests were conducted to evaluate how the system performs under multiple user requests.
The system successfully handles concurrent API requests due to:

- FastAPI asynchronous request handling
- Efficient database queries
- Optimized vector similarity search.

#### 13.5.7 Security Validation

Security testing was performed to ensure that the system protects user data and prevents unauthorized access. Security mechanisms implemented include:

- Password hashing using bcrypt
- JWT-based secure authentication
- Restricted database access
- Secure API communication
- Environment-based configuration for sensitive credentials.

These measures ensure that the ResearchHub AI system operates securely while maintaining data privacy.

# 14 DEPLOYMENT

## 14.1 Deployment Architecture

The deployment architecture of ResearchHub AI follows a **three-tier structure consisting of the presentation layer, application layer, and data layer**. Each layer performs specific functions that collectively enable the complete operation of the system.

**Presentation Layer (Frontend)**

The presentation layer provides the graphical interface through which users interact with the platform. It is implemented using **React and TypeScript**, which enable the creation of dynamic and responsive web interfaces. Through this layer, users can perform activities such as logging into the system, searching for research papers, managing workspaces, and interacting with the AI research assistant. The frontend communicates with the backend server through REST API requests and displays the returned results in an interactive format.

**Application Layer (Backend API)**

The application layer is developed using **FastAPI** and acts as the core processing unit of the system. It handles business logic and manages communication between the frontend, database, and AI services. The backend performs several important operations including user authentication, research paper search, workspace management, and AI-based analysis of research papers. It also connects to external academic APIs such as **arXiv and PubMed** to retrieve research paper metadata. Additionally, the backend processes imported research papers by extracting text, generating embeddings, and preparing the data for AI-based analysis.

**AI Processing Layer**

The AI processing component enables advanced research analysis using a **Retrieval Augmented Generation (RAG)** approach. Research paper content is divided into smaller text segments, and each segment is converted into vector embeddings using **Sentence Transformers**. These embeddings are stored in the database using the **pgvector extension**. When a user asks a question, the system performs a vector similarity search to retrieve the most relevant text segments from stored research papers. These segments are then provided as context to the **Groq Llama-3.3-70B-Versatile language model**, which generates a detailed response based on the retrieved information.

**Data Layer**

The data layer manages all structured and vector-based information required by the platform. The system uses **PostgreSQL with pgvector extension** to store user accounts, workspace details, research paper metadata, chat history, and vector embeddings. The database is deployed using **Docker containers**, which ensures a consistent environment and simplifies database management.

## 14.2 Hosting Platform

ResearchHub AI can be deployed using both local and cloud-based infrastructure depending on project requirements.

The **backend service** can be hosted on cloud platforms such as AWS EC2, Google Cloud Compute Engine, DigitalOcean, or lightweight hosting services like Railway and Render. These platforms provide scalable computing resources, secure API hosting, and environment variable management.

The **frontend application** can be deployed using modern frontend hosting platforms such as Vercel, Netlify, or GitHub Pages. These services offer fast deployment pipelines, CDN-based content delivery, and automatic SSL certificate configuration.

The **database layer** can be hosted either locally using Docker containers or through managed database services such as AWS RDS, Supabase, or Neon PostgreSQL. In the current project implementation, PostgreSQL is deployed locally using **Docker Compose**, which also runs Adminer for database management.

### 14.3 Deployment Steps

The deployment process of ResearchHub AI follows a structured sequence to ensure that all components are correctly configured and connected.

First, the development environment is prepared by installing the required tools such as **Python, Node.js, and Docker**. The project repository is then cloned from the version control system.

Next, the **PostgreSQL database container** is started using Docker Compose. The pgvector extension is enabled, and database tables are verified to ensure that the schema has been correctly created.

After the database setup, the **FastAPI backend server** is configured by creating a Python virtual environment and installing all required dependencies from the requirements.txt file. Environment variables such as the database connection string, Groq API key, and JWT secret key are then configured. The backend server is launched using the Uvicorn server:

```
uvicorn app.main:app –reload
```

Once the backend is operational, the **React frontend application** is started by installing Node dependencies and running the development server:

```
npm install
npm start
```

The frontend runs on port **3000**, while the backend runs on port **8000**.

Finally, the system integration is tested by configuring **CORS settings** to allow communication between the frontend and backend services. The full workflow is verified by testing login functionality, research paper search, workspace management, and AI chatbot responses.

### 14.4 Production Considerations

When deploying ResearchHub AI in a production environment, additional factors must be considered to ensure system stability, security, and scalability.

**Security considerations include:**

- Secure storage of API keys and environment variables
- Password hashing using bcrypt
- Proper JWT authentication and token expiration policies

- HTTPS encryption for secure communication
- Restricting unauthorized database access.

**Performance and scalability measures include:**

- Using asynchronous FastAPI endpoints
- Optimizing database queries and indexing
- Implementing caching for frequently searched papers
- Using background tasks for heavy processing operations
- Deploying backend services behind load balancers.

**Monitoring and reliability measures include:**

- Logging system activities and API requests
- Monitoring server and database performance
- Implementing automated database backups
- Maintaining recovery strategies to prevent data loss.

By incorporating these deployment and production considerations, ResearchHub AI can operate as a reliable and scalable platform for intelligent research paper discovery, management, and analysis.

## 15 PROJECT STRUCTURE

```
researchhub-ai/
├── backend/
│   ├── app/
│   │   ├── main.py              # FastAPI application entry point
│   │   ├── database/
│   │   │   ├── database.py      # SQLAlchemy engine, session, init_db()
│   │   │   ├── models.py        # ORM models (User, Workspace, Paper, …)
│   │   │   └── schemas.py       # Pydantic request / response schemas
│   │   ├── routers/
│   │   │   ├── auth_router.py       # POST /auth/register, /login  GET /auth/me
│   │   │   ├── papers_router.py     # GET /papers/search  POST /papers/import
│   │   │   ├── workspace_router.py  # CRUD /workspaces + notes
│   │   │   └── chat_router.py       # POST /chat/query  GET /chat/messages
│   │   ├── services/
│   │   │   ├── search_service.py    # arXiv + PubMed unified search
│   │   │   ├── embedding_service.py # Sentence Transformer singleton
│   │   │   ├── vector_service.py    # pgvector cosine similarity search
│   │   │   ├── llm_service.py       # Groq Llama 3.3 inference
│   │   │   └── paper_service.py     # Import, workspace assignment, chunking
│   │   ├── agents/
│   │   │   └── research_agent.py    # RAG orchestration agent
│   │   ├── utils/
│   │   │   ├── auth_utils.py        # JWT + bcrypt helpers
```

```
│   │   │   ├── pdf_parser.py       # PyMuPDF text extraction
│   │   │   └── chunking.py         # Overlapping text chunker
│   │   └── jobs/
│   │       └── paper_processing.py  # Background embedding generation thread
│   ├── alembic/
│   │   ├── env.py
│   │   └── versions/001_initial_schema.py
│   ├── alembic.ini
│   ├── requirements.txt
│   └── .env
├── frontend/
│   ├── src/
│   │   ├── App.tsx
│   │   ├── main.tsx
│   │   ├── index.css
│   │   ├── hooks/
│   │   │   └── useAuth.ts          # Auth context + hooks
│   │   ├── services/
│   │   │   └── api.ts              # Axios API client
│   │   ├── types/
│   │   │   └── index.ts            # TypeScript interfaces
│   │   ├── components/
│   │   │   ├── Navbar.tsx
│   │   │   ├── PaperCard.tsx
│   │   │   ├── WorkspaceCard.tsx
│   │   │   ├── ChatMessage.tsx
│   │   │   └── LoadingSpinner.tsx
│   │   └── pages/
│   │       ├── Login/Login.tsx
│   │       ├── SearchPapers/SearchPapers.tsx
│   │       ├── Workspace/Workspace.tsx
│   │       └── Chatbot/Chatbot.tsx
│   ├── package.json
│   ├── vite.config.ts
│   ├── tailwind.config.js
│   └── tsconfig.json
└── docker-compose.yml
```

## 16 RESULTS

- System Output

  The developed system successfully performs the core functionalities defined in the project objectives. The application allows users to interact with the platform through a structured interface and obtain meaningful results.

  Key outputs of the system include:

i. **User Authentication**

   a. Users can register and securely log into the system using JWT-based authentication.

ii. **Research Paper Discovery**

   a. The search module retrieves relevant research papers from academic databases such as arXiv and PubMed.
   b. Results include metadata such as paper title, authors, abstract, and publication date.

iii. **Workspace Management**

   a. Users can create multiple workspaces to organize research papers by topic or project.
   b. Papers can be imported into workspaces with a single click.

iv. **AI-Based Research Assistance**

a. The chatbot analyzes research papers using Retrieval Augmented Generation (RAG).

b. It provides contextual answers to research-related questions by retrieving relevant paper segments.

v. **Semantic Paper Analysis**

a. The system generates embeddings for paper content and performs vector similarity search to identify relevant information before generating AI responses.

These outputs confirm that the platform effectively supports research paper discovery, organization, and analysis.

- Performance Evaluation

The performance of the ResearchHub AI system was evaluated based on system responsiveness, AI analysis efficiency, and overall usability.

Key observations include:

i. **Fast Search Response**

a. Research paper search results are retrieved quickly through optimized API communication with external academic databases.

ii. **Efficient Backend Processing**

a. The FastAPI backend handles user requests asynchronously, ensuring smooth interaction between system components.

iii. **Accurate AI Responses**

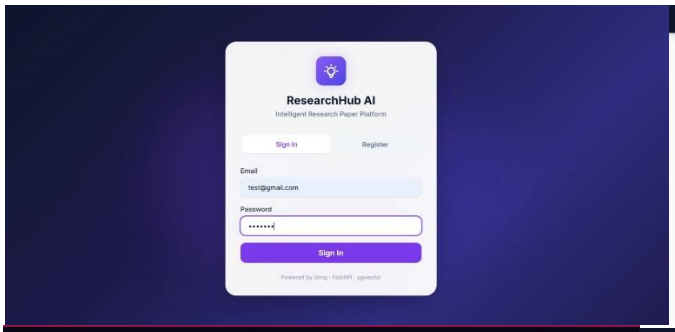
a. The RAG-based AI assistant successfully retrieves relevant document segments before generating responses, improving answer reliability.

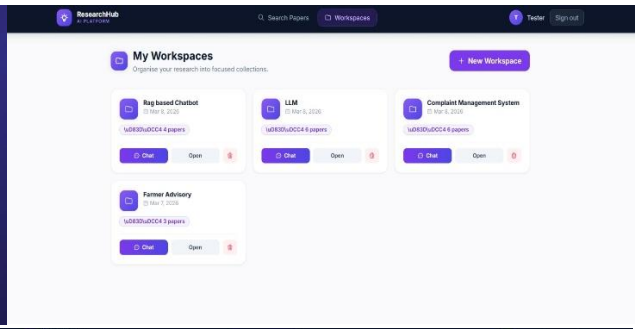
iv. **Scalable Architecture**

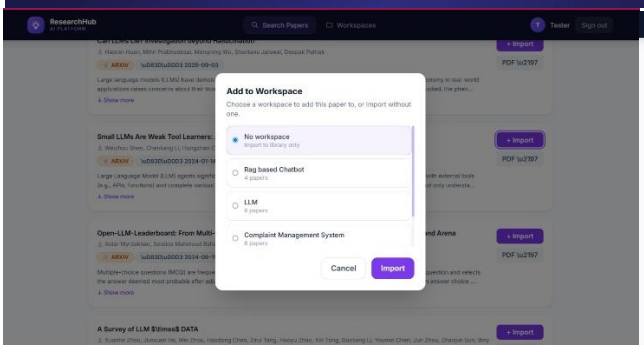
a. The modular architecture enables the system to support multiple users and research queries simultaneously.

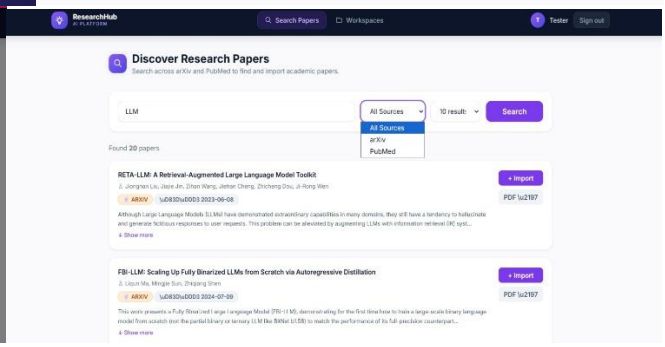
Overall, the system demonstrates reliable performance in handling research queries and managing academic data.

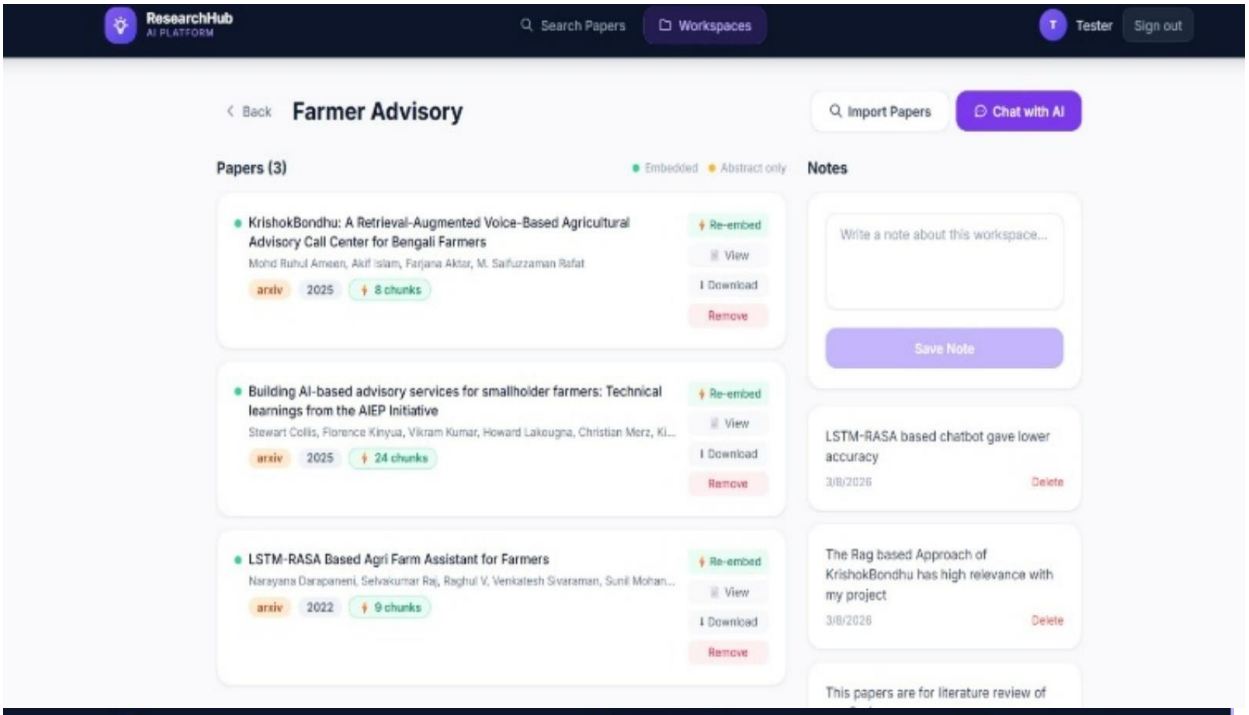
- Screenshots

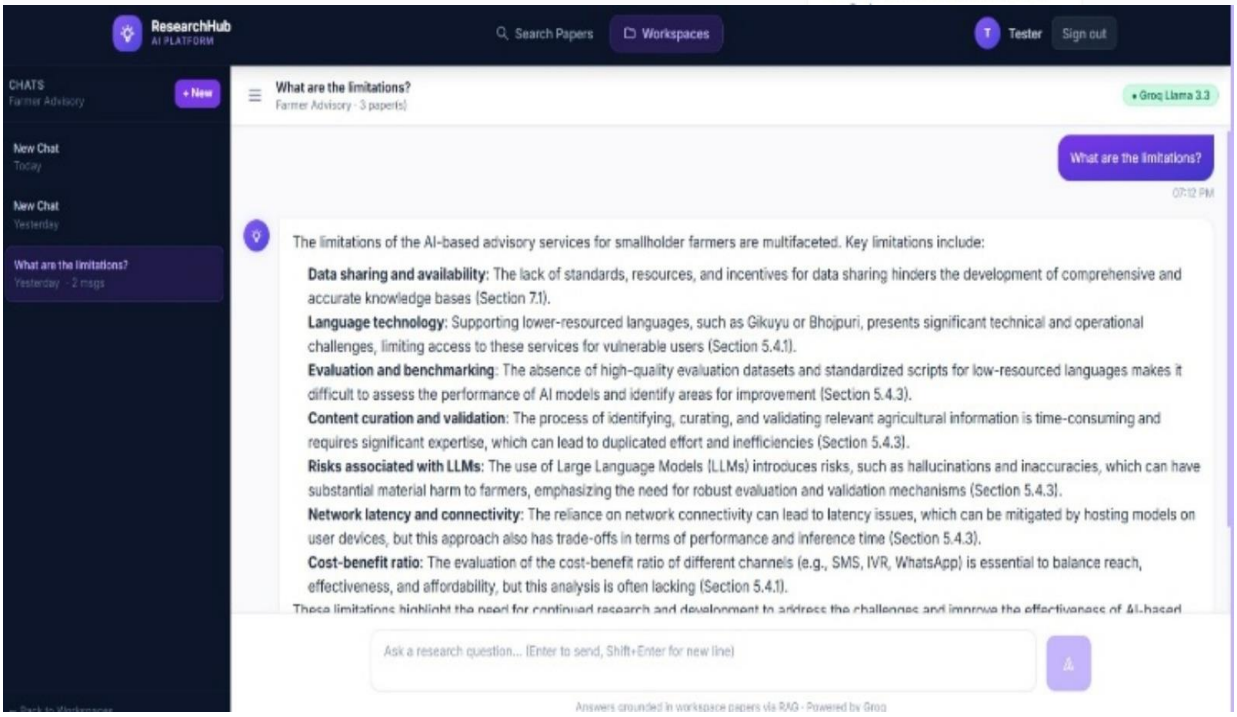





- Benchmark results

  Benchmark testing was conducted to evaluate the system's efficiency and response behavior under normal operating conditions.

  The following observations were recorded:

i. **Average Search Response Time**

   a. Research paper search queries typically return results within a few seconds depending on the external API response time.

ii. **AI Chat Response Time**

   a. AI-generated responses are produced within a short time frame after retrieving relevant document embeddings.

iii. **Database Query Efficiency**

a. PostgreSQL with pgvector enables efficient retrieval of document embeddings through vector similarity search.

iv. **System Stability**

a. The application remains stable during multiple simultaneous requests due to asynchronous API handling and optimized database queries.

These benchmark results demonstrate that the ResearchHub AI platform operates efficiently while maintaining accurate research analysis capabilities.

## 17. ADVANTAGES & LIMITATIONS

### Advantages

The ResearchHub AI system provides several advantages for researchers and academic users. One of the primary advantages is the automation of literature discovery and analysis. By integrating research paper databases and artificial intelligence, the system significantly reduces the manual effort required to identify and interpret relevant academic papers.

Another advantage is the ability to analyze multiple research papers simultaneously. Traditional research workflows require reading each paper individually, whereas the AI assistant can synthesize information from multiple documents and provide summarized insights.

The workspace-based organization system allows researchers to manage multiple research projects independently. Each workspace maintains its own papers, conversation history, and contextual understanding, improving research organization and productivity.

The system also enhances research comprehension by providing contextual explanations and comparisons between papers. This capability is particularly useful for students and early-stage researchers who may find it difficult to interpret complex academic literature.

Finally, the modular architecture of the system makes it easy to integrate additional APIs, AI models, and features in the future.

### Limitations

Despite its advantages, the system also has certain limitations. One limitation is the dependency on external research APIs for retrieving research paper metadata. If these services experience downtime or rate limits, search functionality may be temporarily affected.

Another limitation relates to AI model responses. Although the system uses Retrieval-Augmented Generation to ground responses in research papers, the AI assistant may still produce incomplete or simplified interpretations of complex research findings.

The system also requires an initial embedding generation process for each imported paper, which may take additional processing time before the paper becomes available for AI analysis.

Furthermore, the system currently operates as a web-based application and does not yet provide dedicated mobile applications or offline capabilities.

Addressing these limitations will be an important consideration in future system improvements.

## 18. FUTURE ENHANCEMENTS

### Scalability Improvements

Future versions of the system can introduce enhanced scalability mechanisms to support larger research datasets and a greater number of concurrent users. Distributed database architectures and cloud-based vector search systems can improve performance when processing large collections of research papers. Additionally, asynchronous task processing frameworks can be used to handle document parsing and embedding generation more efficiently.

### Feature Expansion

The system can be extended with additional features to enhance research analysis capabilities. For example, automated citation extraction and reference graph visualization could help users understand relationships between research papers. Advanced recommendation systems could also suggest relevant papers based on user research interests and previous queries.

### Cloud Integration

Integrating the platform with cloud computing services would allow large-scale deployment and improved resource management. Cloud platforms such as AWS, Google Cloud, or Azure can be used to host the application, manage document storage, and scale AI inference workloads dynamically. This would enable the system to support institutional research environments and large collaborative research teams.

### Mobile Integration

Developing mobile applications for the platform would allow researchers to access their workspaces and AI assistant through smartphones or tablets. Mobile integration would enable users to quickly review research papers, view summaries, and ask contextual questions while working remotely or attending conferences.

### Automation

Future development may also include advanced automation capabilities such as automated research monitoring. The system could automatically track new publications related to a user's research topics and notify them when relevant papers are published. Automated summarization pipelines and scheduled research reports could further assist researchers in staying updated with developments in their field.

## 19. CONCLUSION

### Summary of Implementation

The ResearchHub AI project successfully demonstrates the development of an intelligent research assistance platform that combines web technologies, database systems, and artificial intelligence. The system integrates research paper discovery, document management, and AI-based analysis into a unified environment that supports efficient research workflows.

Through the integration of modern technologies such as React, FastAPI, vector databases, and large language models, the platform provides a scalable architecture capable of handling research data and generating contextual insights.

### Technical Achievements

The project achieves several technical milestones including the implementation of a Retrieval-Augmented Generation architecture for contextual AI responses, integration with external research databases, and the use of vector embeddings for semantic document retrieval. The use of Groq's Llama 3.3 model enables fast and accurate natural language responses based on research paper content.

The system also demonstrates effective use of modern software engineering practices including modular backend design, API-based communication, and containerized deployment environments.

### Practical Impact

The ResearchHub AI platform provides a practical solution to the growing challenge of managing and analyzing large volumes of academic literature. By automating research discovery and enabling contextual AI analysis, the system reduces the time required for literature review and improves the efficiency of research workflows.

Such platforms have the potential to transform how researchers interact with scientific knowledge, enabling faster insights, improved collaboration, and more informed research decision-making. The system therefore represents an important step toward intelligent research management tools that support the evolving needs of modern scientific research.

# 17 APPENDIX

- **Source Code**

- **Configuration Files**

**Backend Dependencies (requirements.txt)**

The backend of the ResearchHub AI system is developed using Python and FastAPI. The following dependencies are defined in the requirements.txt file to install the required libraries for the application.

These packages support API development, database connectivity, authentication, AI model integration, document processing, and vector-based retrieval.

**Installation Command**:
pip install -r requirements.txt

**requirements.txt:**

```
fastapi==0.115.0
uvicorn[standard]==0.32.0
sqlalchemy==2.0.36
psycopg2-binary==2.9.10
alembic==1.14.0
python-dotenv==1.0.1
bcrypt>=4.0.1
python-jose[cryptography]==3.3.0
groq==0.13.0
sentence-transformers==3.3.1
pymupdf==1.25.0
requests==2.32.3
feedparser==6.0.11
pydantic==2.10.3
pydantic-settings==2.7.0
chromadb>=1.5.0
httpx==0.28.1
python-multipart==0.0.20
numpy>=2.0.0
```

**Purpose of Dependencies**

- **FastAPI, Uvicorn** – Backend API framework and server
- **SQLAlchemy, psycopg2, Alembic** – Database connection and migrations
- **python-jose, bcrypt** – Authentication and security
- **Groq SDK** – Integration with the Llama AI model
- **Sentence Transformers, ChromaDB** – Embedding generation and vector retrieval
- **PyMuPDF** – PDF text extraction
- **Requests, Feedparser, HTTPX** – External API communication
- **Pydantic** – Data validation and configuration management

- **API Documentation**

**1) Groq API – AI Research Assistant**

The Groq API is used to integrate a large language model for generating AI responses to user research queries.
Configuration
- API key stored in environment file: backend/.env
- Required variables:
  - GROQ_API_KEY
  - SECRET_KEY
  - DATABASE_URL
- .env file is excluded from version control using .gitignore.

Dependencies
- groq
- python-dotenv

Model Used
- llama-3.3-70b-versatile

Backend Endpoint
- POST /chat

Functionality
- Accepts user queries from the frontend.
- Sends the query to the Groq language model.
- Returns AI-generated responses based on research context.

Workflow
1. User submits query in chatbot interface
2. Frontend sends request to /chat endpoint
3. Backend processes request and prepares context
4. Query sent to Groq API
5. AI-generated response returned to frontend

**2) arXiv API – Research Paper Retrieval**

The arXiv API is used to retrieve research papers related to user queries.
Key Characteristics
- Public API (no API key required)
- Accessed through HTTP requests

Base Endpoint
http://export.arxiv.org/api/query
Common Parameters
- search_query – research keyword
- start – starting result index
- max_results – number of papers returned

Dependencies
- requests
- feedparser

Functionality
- Retrieves research paper metadata including:
  - Paper title
  - Authors
  - Abstract
  - Publication link

Backend Endpoint
- GET /search

**3) PubMed API – Biomedical Research Retrieval**

The PubMed API is used to retrieve biomedical research papers from the NCBI database.
Configuration

- API key stored in backend/.env:
  - PUBMED_API_KEY

Dependencies

- requests
- biopython

Primary Endpoint
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi
Functionality

- Searches biomedical literature using NCBI E-utilities
- Retrieves PubMed IDs for matching research papers

Backend Endpoint

- GET /search

Rate Limits

- Without API key: 3 requests per second
- With API key: 10 requests per second

- **GitHub Repository Link**

  https://github.com/Khilesh-01/Research-Hub-AI.git

- **Live Demo Link**

  https://drive.google.com/drive/folders/1dtP_hYTk2EhyX53hVI8JPOODynaniFVu